# Full-Stack Web Development — Day 11 Study Material

**MODULE**
Backend Foundations

**SUBJECT**
NestJS Architecture & First API

**AUTHOR**
Penumuru Madhu Sudhan Reddy

**Executive Summary:** Introduction to enterprise server-side development with NestJS. Covers three-tier architecture (Modules, Controllers, Providers), Dependency Injection (DI), in-memory CRUD operations, hierarchical REST routing (Projects & Towers), and Postman collection testing.

## 1. Express vs. NestJS: The Enterprise Shift

| Feature | Vanilla Express | NestJS |
|---|---|---|
| **Architecture** | Unopinionated, free-form | Structured, three-tier modular pattern |
| **Dependency Injection** | Manual parameter passing | Built-in IoC Container via constructor DI |
| **Decorators** | None (manual router middleware) | Declarative (`@Get`, `@Post`, `@Param`) |
| **TypeScript** | Optional, manual types | First-class citizen with strict type safety |

## 2. The Three-Tier Architecture

NestJS organizes code into three distinct, decoupled layers:

- **Modules (`@Module`):** Encapsulate closely related business domains (e.g. `ProjectsModule`).
- **Controllers (`@Controller`):** Handle HTTP routing, parse incoming parameters, and delegate work to services.
- **Providers / Services (`@Injectable`):** House business logic and data persistence, injected via constructor.

## 3. CRUD API & Nested Resources (Exercises 2 & 4)

```
// Projects Controller CRUD Endpoints
@Controller('projects')
export class ProjectsController {
  constructor(private readonly service: ProjectsService) {}

  @Get() getAll() { return this.service.findAll(); }
  @Get(':id') getOne(@Param('id') id: string) { return this.service.findOne(id); }
  @Post() create(@Body() data) { return this.service.create(data); }
  @Put(':id') update(@Param('id') id: string, @Body() data) { return this.service.update(id, data); }
  @Delete(':id') remove(@Param('id') id: string) { return this.service.remove(id); }
}

// Nested Tower Controller
@Controller('projects/:id/towers')
export class TowersController {
  constructor(private readonly towersService: TowersService) {}
  @Get() getTowers(@Param('id') id: string) { return this.towersService.findByProject(id); }
}
```

```
Endpoint Verification (o/p):
- GET /projects → Status 200 OK (Returns 3 projects)
- POST /projects → Status 201 Created (Generates proj-172895... ID)
- GET /projects/:id/towers → Status 200 OK (Returns nested tower array)
- GET /projects/invalid-id → Status 404 Not Found ("Project not found.")
```